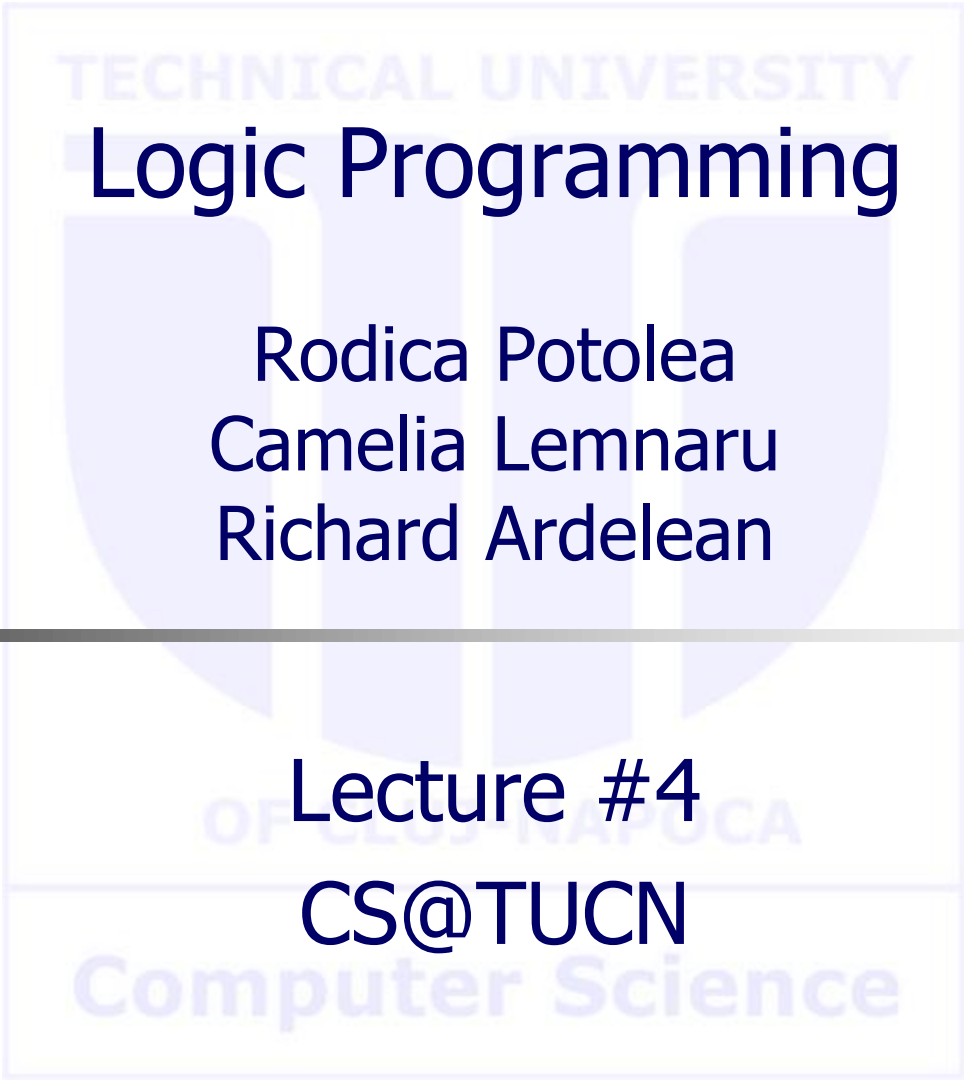




Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #4
CS@TUCN

Agenda

- **Forward vs backward recursion**
 - sum
 - generics
 - Comparative analysis, Pros and cons
- **append3**
 - Forms
 - Efficiency analysis and justification
 - Nondeterministic call
- **Delete from a list**
 - One, just one, all, repeating the query

Forward vs backward recursion

- Forward vs backward - alternative approaches to handle data
- Data is split into partitions
- Forward = processing takes place when the current item is processed when is first encountered, and the rest of the data (the other components than the item) are processed **AFTER** the current item is processed.
- Backward = starts by processing all next after current item (the other components than the item) via recursive call(s) while the current item is processed just **AFTER** we return from recursion(s).
 - *NOTE:* it is returned from recursive call (and **NOT from backtracking!!!**).
- In case of lists [H|T]:
 - *forward* handles H first and next T (via recursive call),
 - *backward* starts by solving the problem on T (via recursive call), and just when returning from the call H is processed.

Forward sum

```
%sum2/3 (In_list, Final_result, Accumulator_partial_result).  
  
sum2([], PartialSum, PartialSum).    % final result, copies the value of the partial  
                                     % result with default unification  
  
sum2([H|T], Sum, PartialSum) :-  
    NewPartialSum is PartialSum + H,    % do process the current item  
    sum2(T, Sum, NewPartialSum).        % go ahead with the remaining struct
```

- List decomposed into [H|T]
 - Starts by processing (addition here) the current item (H here)
 - Continue with processing the rest of the structure (one recursive call here, as partition is H and T)
 - This implies the items are added in the sum forward (starting from the first one).
- ->->->...-> processing is performed in the order of items in the structure
- The accumulated result represents the sum of values from the beginning to the current item

Backward sum

```
% sum1/2 (In_list, Sum_of_els_in_arg_1).
```

```
sum1([],0). % result gets initialized. Empty input, null
sum1([H|T],Sum):-
    sum1(T,TailSum), % call processing first on rest of the partition
    Sum is TailSum + H. % do process the current item
```

- List decomposed into [H|T]
 - Starts with processing T, same predicate, recursive call
 - When returned, use the obtained result ($TailSum$) to evaluate the overall result on the whole list (Sum)
 - This implies the items are added in the sum backwards (starting from the last one).
- ---... --> first go this way (all way to the end) doing nothing (just call)
- <- the processing occurs after returning from the call, one at a time
 - in the opposite order, *last* item *first* processed, before last item second processed,
 - third, second and first items being last processed in this specific order (first is last)
- The accumulated result is the sum of values from the current item to the end

Forward vs backward sum

- Forward solution needs specific initial call
 - write a special predicate (a wrapper) to make it in a sound way (avoid the need of user knowing how to initialize the accumulator parameter):

```
sum2(List, Sum) :-                % same partial result as in case of backward
    sum2(List, Sum, 0).           % nothing yet processed, null result.
                                % Same as in stop condition for bwd.
```

- If the input list is [1,2,3,4,5,6,7] what is going to be the partial result (PartialSum) and (TailSum) respectively on forward and backward solutions?
- Try to **estimate before running** them.
- Follow the textbook to update the predicate with the necessary lines to print the values.

<pre>% sum1/2 (List, Result). sum1([], 0). sum1([H T], Sum) :- sum1(T, TailSum), Sum is TailSum + H.</pre>	<pre>%sum2/3 (List, Result, Accumulator). sum2([], PartialSum, PartialSum). sum2([H T], Sum, PartialSum) :- NewPartialSum is PartialSum + H, sum2(T, Sum, NewPartialSum).</pre>
--	---

Forward generic

```
% forward_recursion/3 (input argument, final result, partial result)

% final result(arg 2) copies partial result(arg3) value with default unification
forward_recursion([], PartialResult, PartialResult).

% partition data, here split into H and T
forward_recursion([H|T], Result, PartialResult) :-
    % start by processing the current item and thus
    % updating previous PartialResult to NewPartialResult via processing do
    do(NewPartialResult, H, PartialResult),

    % process the rest of the structure with recursive call.
    forward_recursion(T, Result, NewPartialResult).

%forward_recursion_call/2 (in, out)
forward_recursion_call(Input, Output) :-
    forward_recursion(Input, Output, InitialValueOfResult)
% make the initialization with a separate predicate (wrapper)
% to avoid mandatory user initialization
```

Backward generic

```
% backward_recursion/2 (input argument, output result)
```

```
% empty input, make initialization backwards
```

```
backward_recursion([], InitialValue) .
```

```
backward_recursion([H|T], PartialResult) :-
```

```
    % starts with processing the rest of the structure;
```

```
    % all partition but the current item.
```

```
    backward_recursion(T, NewPartialResult),
```

```
    %process the current item
```

```
    do(PartialResult, H, NewPartialResult) .
```

- No need for a specific initial call, hence, no wrapper predicate.

Forward vs backward pros and cons

	Forward	Backward
+	• Process "as we go" => structure is processed from front to end => intermediate results could be useful (the result of the structure "so far"). • In concurrent processing that result is made available and another process using it can start immediately • Last Call Optimization = reusing the same stack area without the need of restoring it back	• Needs no initialization => • Needs no specific call => • needs no wrapper => • needs no additional argument
-	• Needs specific initial call => always make a wrapper to initialize the accumulator • Needs additional argument (partial result)	• Intermediate results are seldom useful • The result of the structure is known just at the end of processing, so, concurrency is postponed on sync

Concatenate 3 lists

- Use what you ***have*** vs use what you ***know***
- We have the concatenation of 2 lists
- Use it twice.

```
append([], List, List).  
append([Head|Tail], List, [Head|Rest]) :-  
    append(Tail, List, Rest).
```

- To put together L1, L2 and L3 do the following
 - $(L1+L2) + L3$ OR $L1 + (L2+L3)$

Concatenate 3 lists: Use what you have 1

```
append([], L, L) .
append([H|T], L, [H|R]) :-
    append(T, L, R) .
```

- $(L1+L2) + L3$; say $|L1|=n1$, $|L2|=n2$, $|L3|=n3$

% Efficiency: $O(n)$ where n the length of the first parameter

```
append3_1(L1,L2,L3,Result) :
```

```
    append(L1,L2,Intermediate) ,
```

% link L2 at the end of L1 = Intermediate

```
    append(Intermediate,L3,Result) .
```

% link L3 at the end of the intermediate result

- Efficiency:
 - $O(n1)$ for the 1st call (links $L2$ at the end of $L1$)
 - $O(n1+n2)$ for the 2nd call, length $Intermediate$ of is length of $L1$ and $L2$ (links $L3$ at the end of $Intermediate$)
 - Overall: $t(n)=2n1+n2$

Concatenate 3 lists: Use what you have 2

```
append([], L, L) .
append([H|T], L, [H|R]) :-
    append(T, L, R) .
```

- $L1 + (L2 + L3)$; say $|L1|=n1$, $|L2|=n2$, $|L3|=n3$

```
append3_2(L1, L2, L3, Result) :-
    append(L2, L3, Intermediate),           % link L3 at the end of L2 = Intermediate
    append(L1, Intermediate, Result) .      % link intermediate at the end of the first list L1
```

- Efficiency:
 - $O(n2)$ for the 1st call, decomposes $L2$ (links $L3$ at the end of $L2$)
 - $O(n1)$ for the 2nd call, decomposes $L1$ (links $Intermediate$ at the end of $L1$)
 - Overall: $t(n)=n1+n2$
- Observations:
 - **Second version better regardless the input!**
 - Order of calls matters (again!)
 - How can we use this in a standalone predicate?

Concatenate 3 lists: Use what you know

- What do we know?
 - Concatenation via decomposition of the first argument! Use it!

```
append3_3([H|T], L2, L3, [H|R]) :-  
    % as long as the first arg  
    % nonempty, decompose it  
    append3_3(T, L2, L3, R).  
append3_3([], [H|T], L, [H|R]) :-  
    % once first argument empty  
    % you are back on 2 list concatenation  
    append3_3([], T, L, R).  
append3_3([], [], L, L).
```

- Observations:
 - Clauses 2 and 3 are disjoint from clause 1 (indexation on the first argument would treat them separately, i.e. cl1 one entrance, cl 2 and 3 another one). Therefore, it does NOT matter where clause 1 is placed
 - On the other hand, clause 3 should come AFTER clause 2 (as indexation on the second argument is NOT available)

Concatenate 3 lists: Use what you know – contd.

```
append3_3([H|T], L2, L3, [H|R]) :-
    append3_3(T, L2, L3, R) . % decomposes first list
append3_3([], [H|T], L, [H|R]) :-
    append3_3([], T, L, R) . % decomposes second list
append3_3([], [], L, L) .
```

• Observations:

- How is it done? (resembles behavior of version1)
 - Decomposes L_1 to traverse it and adds each item in result (behaves as version 1, as in "link L_2 at the end of L_1 ")
 - Decomposes L_2 to traverse it and adds each item in result (links L_3 at the end of L_1 concatenated to L_2).
- Efficiency (resembles in performance to version2) $t(n)=n_1+n_2$
 - $O(n_1)$ first clause
 - $O(n_2)$ second clause

```
append3_2(L1, L2, L3, Result) :-
    append(L2, L3, Intermediate) , % link L3 at the end of L2 = Intermediate
    append(L1, Intermediate, Result) . % link intermediate at the end of the first list L1
```

Concatenate 3 lists: Use what you know – contd.

- Behavior: resembles to version1 $(L1+L2) + L3$
- Efficiency: resembles to version2 $t(n)=n1+n2$
- How is possible? Behavior V1 and performance V2?
- Explain!
- Explain which of the 3 versions is best?
- Which to use and why?
- Which should never be used? Why?

Concatenate 3 lists: Nondeterministic call

- Given `append3` predicate and the call: `?-append3(X,Y,Z,[1,2])`.
- What is the meaning of the call?
 - Nondeterministic call
 - Which lists `X`, `Y` and `Z` concatenated form list `[1,2]`.

- What is/are the result/s?

X	Y	Z
[]	[]	[1,2]
[]	[1]	[2]
[]	[1,2]	[]
[1]	[]	[2]
[1]	[2]	[]
[1,2]	[]	[]

- Other results? Why not?
- Which order/why?
 - Depends on the implementation.
 - Identify (BEFORE running) the order of results in each implementation
 - For `append3` with `append`, the `append` order of clauses is MANDATORY (fact first), otherwise it enters infinite loop with 3 free variables on the first call.

Delete from a list

- Given a list, remove one item from it.
- How many arguments/why?

```
%delete/3 (Item_to_remove, Input_list, Output_list)
```

```
delete(H, [H|T], T) .      % if item to remove = head of input, don't put it in output  
delete(X, [H|T], [H|R]) :- % otherwise, keep it on output and  
    delete(X, T, R) .      % remove from tail
```

- What are the results on call when the item occurs several times? Try to estimate and justify BEFORE running.
 - [q1] ?- delete(3, [1, 3, 2, 3, 4, 3], Output) .
- What happens in case the item is not present in the list?
 - [q2] ?- delete(5, [1, 3, 2, 3, 4, 3], Output) .

Delete from a list – contd.

```
delete(H, [H|T], T) .
```

```
delete(X, [H|T], [H|R]) :-  
    delete(X, T, R) .
```

- What are the results on call? Why?

```
| ?- delete(1, [1, 2, 1, 3, 1], R) .
```

```
R = [2, 1, 3, 1] ? ;
```

```
R = [1, 2, 3, 1] ? ;
```

```
R = [1, 2, 1, 3] ? ;
```

```
no.
```

- What about the call? Why?

```
| ?- delete(4, [1, 2, 1, 3, 1], R) .
```

```
no.
```

So, the meaning of the predicate is:

- **delete exactly one occurrence** of an item from the list.

Delete from a list - contd

```
delete(H, [H|T], T) .      % if item to remove = head of input, don't put it in output
delete(X, [H|T], [H|R]) :- % otherwise, keep it on output and
    delete(X, T, R) .      % remove from tail
delete(_, [], []).
% when the empty list is reached,
% whatever element is assumed to
% be deleted, done, result empty
```

- What are the results on call when the item occurs several times? Try to estimate and justify **BEFORE running**.
- What happens in case the item is not present in the list?

Computer Science

Delete from a list - contd

```
delete(X, [X|T], T) .  
delete(X, [H|T], [H|R]) :-  
    delete(X, T, R) .  
delete(_, [], []) .
```

- What are the results on call?

```
| ?- delete(1, [1,2,1,3,1], R) .
```

```
R = [2,1,3,1] ? ;
```

```
R = [1,2,3,1] ? ;
```

```
R = [1,2,1,3] ? ;    % what's next? Why?
```

```
R = [1,2,1,3,1] ? ;
```

```
no.
```

- What about the call? Why?

```
| ?- delete(4, [1,2,1,3,1], R) .
```

```
R = [1,2,1,3,1] ? ;
```

```
no.
```

So, the meaning of the predicate is:

- **delete one occurrence** of an item from the list;
- if **absent**, do **NOTHING** (leave the list unchanged).

Delete from a list - contd

- What are the differences when the 2 implementations (without/with 3rd clause) are compared? Explain!
- What happens if the clause when item is found cuts the backtrack?
- Implementation without 3rd clause:

```
delete(H, [H|T], T) :- ! .  
delete(X, [H|T], [H|R]) :-  
    delete(X, T, R) .
```

- A cut in a clause is as if in all consequent clauses we add the negation of the conjunction to the left of the cut. Therefore, in a clause like:
 - $p:-q,r,! ,s.$
 - The cut implies a “default” negation of q,r (as $\text{not}(q,r)$) in all clauses after it.
- In the predicate `delete`, first clause contains **nothing** to the left of the cut. So, what does it negate?
- Does it make any sense to place that cut?

Delete from a list - contd

```
delete(H, [H|T], T) :- !.    % means: delete(X, [H|T], T) :- X=H, !,  
delete(X, [H|T], [H|R]) :-  % therefore a X!=H here  
    delete(X, T, R).
```

- In the implementation with 3rd clause:

```
delete(H, [H|T], T) :- !.  
delete(X, [H|T], [H|R]) :-  
    delete(X, T, R).  
delete(_, [], []).
```

- Answer the same queries for **both** implementations and estimate the output (initial queries and repetitions). Explain!

```
| ?- delete(1, [1,2,1,3,1], R1).
```

```
R1=...? ; ? WHY?
```

```
| ?- delete(4, [1,2,1,3,1], R2).
```

```
R2=...? ; ? WHY?
```

Delete all occurrences of an item from a list

Start from the first version of the predicate

```
delete(H, [H|T], T) .  
delete(X, [H|T], [H|R]) :-  
    delete(X, T, R) .
```

```
delete(H, [H|T], R) :- % if item to remove = head of input, don't put it in output  
    delete(H, T, R) . % but also remove other occurrences from tail  
delete(X, [H|T], [H|R]) :- % otherwise, keep it on output and  
    delete(X, T, R) .      % remove from tail
```

- Could it be just this?
- Justify!

Delete all occurrences of an item from a list

% if the item to remove is head of input, don't put it on output

```
delete_all(H, [H|T], R) :-
```

```
    % but also remove other occurrences from tail
```

```
    delete_all(H, T, R).
```

% otherwise, keep it on output and

```
delete_all(X, [H|T], [H|R]) :-
```

```
    % remove from tail
```

```
    delete_all(X, T, R).
```

% when the empty list is reached, done, result empty

```
delete_all(_, [], []).
```

```
?- delete_all(1, [1,2,1,3,1], R).
```

```
R = [2,3]
```

- What happens if we repeat the query? Why?
- How many answers? Which order? Explain!
- How can we obtain just the answer from above?

Delete all occurrences of an item from a list

```
delete_all(H, [H|T], R) :-  
    !,  
    delete_all(H, T, R).  
delete_all(X, [H|T], [H|R]) :-  
    delete_all(X, T, R).  
delete_all(_, [], []).
```

- A cut in a clause is as if in all consequent clauses we add the negation of the conjunction to the left of the cut. Therefore, in a clause like:
 - $p:-q,r,!s$.
 - The cut implies a “default” negation of q,r (as $\text{not}(q,r)$) in all clauses after it.
- In the predicate `delete_all`, first clause contains **nothing** to the left of the cut. So, what does it negate?
- Does it make any sense to place that cut?

Delete all occurrences of an item from a list

```
delete_all(H, [H|T], R) :- !,  
    delete_all(H, T, R).
```

What does ! negates?

Is the default unification! The clause above is like:

```
delete_all(X, [H|T], R) :-  
    X=H, !,  
    delete_all(X, T, R).
```

Therefore, the cut negates **X=H**, thus, in the next clauses, there is a default **X≠H**.

Delete all occurrences of an item from a list

```
delete_all(H, [H|T], R) :-  
    !,  
    delete_all(H, T, R).  
delete_all(X, [H|T], [H|R]) :-  
    delete_all(X, T, R).  
delete_all(_, [], []).  
  
?- delete_all(1, [1, 2, 1, 3, 1], R).
```

- What is the result? Why?
- What happens if we repeat the query? Why? How many answers? Which order? Explain!

Conclusion

- Order of clauses matters
- Order of calls matters
- Always estimate performance
- Meaning of
 - Repeating the query
 - The cut
- Questions?

References explained

1. **Lloyd**, J. W, Foundations of Logic Programming, Springer, 1984
2. **Clocksin**, William, **Mellish**, Christopher S., Programming in Prolog - Using the ISO Standard, Springer 1981
3. Leon S. **Sterling** and Ehud Y. **Shapiro**, The Art of Prolog Advanced Programming Techniques
4. Robert **Kowalski**, Logic for Problem Solving, 1979
5. Rodica **Potolea**, Programare Logica (vol 1), UTPress, 2007